

Tutorial 12: Cache Design Problems & Solutions

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

Problem 1

Consider a processor with 32-bit byte-addressable memory and a 64 KiB (65,536 bytes) cache. The cache block size is 64 bytes. Answer the following parts.

Part (a) – Direct-Mapped Cache

For a direct-mapped cache:

- i. Determine the number of cache lines (blocks).
- ii. Calculate the number of bits needed for the block offset, index, and tag.
- iii. If a memory address 0x00F4A3C8 is referenced, what are its tag, index, and offset values (in binary or hex)?

Part (b) – Tag Directory Overhead

For the direct-mapped cache from part (a), compute the total number of tag bits stored in the cache (excluding valid/dirty bits). Also, what is the total cache size, including tags and data (assuming 1 valid bit per block, no dirty bit)?

Part (c) – 4-Way Set-Associative Cache with LRU

Now assume the same total cache size (64 KiB) and block size (64 bytes), but reorganized as a 4-way set-associative cache.

- i. How many sets are there?
- ii. How many bits are needed for offset, set index, and tag?
- iii. Given the following sequence of memory address accesses (in decimal, for simplicity), determine the number of cache hits using an LRU replacement policy. Show the state of the cache after each access (only the tags in each set).

Addresses: 1024, 1088, 1152, 1088, 1216, 1024, 1088

Assumption: Addresses are **byte addresses** (decimal). Block size = 64 bytes. Block number = address $\div$ 64; set index = block number mod number of sets.

Part (d) – Fully Associative Cache Comparison

If the cache were fully associative (same 64 KiB total size, 64-byte blocks):

- i. How many blocks would the cache have?
- ii. How many bits are needed for the tag and offset? Why is no index needed?
- iii. For the same address sequence as in part (c), would the hit count change? Briefly explain why?

Solution – Problem 1

Solution Part (a) – Direct-Mapped Cache

- i. Lines = $65536/64 = \mathbf{1024}$.
- ii. Bit fields (32-bit address):
 - Offset: $\log_2 64 = \mathbf{6}$ bits
 - Index: $\log_2 1024 = \mathbf{10}$ bits
 - Tag: $32 - 10 - 6 = \mathbf{16}$ bits
- iii. Address 0x00F4A3C8 in binary:

$$\underbrace{0000\ 0000\ 1111\ 0100}_{\text{tag}=0x00F4} \underbrace{1010\ 0011\ 11}_{\text{index}=0x28F=655} \underbrace{00\ 1000}_{\text{offset}=8}$$

Answer: Tag = 0x00F4, Index = 655₁₀, Offset = 8

Solution Part (b) – Tag Directory Overhead

Tag bits stored: $1024 \times 16 = 16384$ bits = **2 KiB**

Total size (data + tag + 1 valid bit per line):

$$1024 \times (512 + 16 + 1) = 1024 \times 529 = 541,696 \text{ bits} \approx \mathbf{66.1 \text{ KiB}}$$

Answer: Tag storage = 2 KiB; Total (incl. tags + valid) $\approx$ 66.1 KiB

Solution Part (c) – 4-Way Set-Associative, LRU

- i. Sets = $1024/4 = 256$.
- ii. Offset = 6; index = $\log_2 256 = 8$; tag = $32 - 8 - 6 = 18$ bits.
- iii. Block number = $\text{addr}/64$; set = $\text{block} \bmod 256$:

Access	Addr	Block	Set	Result	Cache state
1	1024	16	16	Miss	S16:{16}
2	1088	17	17	Miss	S17:{17}
3	1152	18	18	Miss	S18:{18}
4	1088	17	17	Hit	S17:{17}
5	1216	19	19	Miss	S19:{19}
6	1024	16	16	Hit	S16:{16}
7	1088	17	17	Hit	S17:{17}

Answer: 3 hits (accesses 4, 6, 7); 4 misses

Solution Part (d) – Fully Associative

- i. Blocks = $65536/64 = 1024$.
- ii. Offset = 6 bits; Tag = $32 - 6 = 26$ bits; Index = 0 bits. No index is needed because any block can map to any cache line – the tag alone identifies the block.
- iii. With 1024 slots and only 4 unique blocks accessed, there are no evictions regardless of policy. Hit count = 3, same as 4-way SA.

Answer: 1024 blocks; tag = 26 bits; no index. Hit count unchanged at 3.

Problem 2

Consider a machine with a byte-addressable main memory of 2^{16} bytes. Assume that a direct-mapped data cache consisting of 32 lines of 64 bytes each is used in the system. A 50×50 two-dimensional array of bytes is stored in the main memory starting from memory location 1100H. Assume that the data cache is initially empty. The complete array is accessed twice. Assume that the contents of the data cache do not change between the two accesses. How many data misses will occur in total?

Assumption: The array is stored and accessed in **row-major order** with **sequential traversal** (row by row, left to right) – standard C memory layout.

Solution – Problem 2

Step 1: Cache parameters.

- Lines = 32, block size = 64 bytes, total capacity = 2048 bytes
- Address bits: 16 total; offset = $\log_2 64 = 6$; index = $\log_2 32 = 5$; tag = 5 bits

Step 2: Array layout.

$50 \times 50 = 2500$ bytes starting at $1100H = 4352_{10}$.

First block = $\lfloor 4352/64 \rfloor = 68$, Last block = $\lfloor (4352 + 2499)/64 \rfloor = \lfloor 6851/64 \rfloor = 107$

Total blocks spanned = $107 - 68 + 1 = 40$ blocks.

Step 3: Cache line mapping.

Line = block mod 32. Blocks 68–107 map to lines 4–31, 0–3, then 4–11 again (40 blocks into 32 lines means 8 lines are re-used). Specifically: blocks 68–75 → lines 4–11; blocks 100–107 → same lines 4–11 (conflict!).

Step 4: First access (cold cache).

Every new block → compulsory miss.

$$\text{Misses}_1 = 40$$

Step 5: Second access (conflict analysis).

After the first pass, lines 4–11 hold blocks 100–107 (they overwrote 68–75 midway through). Lines 12–3 (i.e. 12–31, 0–3) hold blocks 76–99 – *still valid*.

Second pass (blocks 68–107 in order):

- Blocks 68–75 (lines 4–11): each hits a line holding block +32 → **8 misses**
- Blocks 76–99 (lines 12–3): still in cache → **24 hits**
- Blocks 100–107 (lines 4–11): now holding 68–75 again (loaded above) → **8 misses**

$$\text{Misses}_2 = 8 + 8 = 16$$

Step 6: Total.

$$\text{Total misses} = 40 + 16 = 56$$

Answer: Total data misses = 56

Problem 3

A 32-bit byte-addressable processor has a **32 KiB cache**, block size **64 bytes**. The system uses **write-back**, **write-allocate**.

(a) Direct-Mapped Cache

- i. Determine the number of lines, tag/index/offset bits, and total tag bits stored.
- ii. If the cache is initially empty, and the CPU accesses the following addresses (in hex):

0x00010004, 0x00010044, 0x00010084, 0x00020004, 0x00010004, 0x00010044

Count the number of hits and misses.

(b) 4-Way Set-Associative Cache (LRU)

Using the same total cache size (32 KiB) and block size (64 bytes), but reorganized as a 4-way set-associative cache with LRU replacement:

- i. Find the number of sets, tag/index/offset bits, and total tag bits.
- ii. Repeat the same address trace. How many hits occur now?

(c) Tag Overhead Comparison

Compare the tag directory overhead (bits only, excluding valid/dirty bits) for both cache configurations.

(d) AMAT Calculation

If the miss penalty is 200 cycles and the hit time is 2 cycles, compute the Average Memory Access Time (AMAT) for both caches using the miss rates obtained from the address trace (assume no other traffic). Which cache design performs better?

Solution – Problem 3

Solution (a) – Direct-Mapped

- i. Lines = $32768/64 = 512$; offset = 6 bits; index = $\log_2 512 = 9$ bits; tag = $32 - 9 - 6 = 17$ bits.

Total tag bits = $512 \times 17 = 8704$ bits.

- ii. Address bit layout: [31:15] tag (17 b), [14:6] index (9 b), [5:0] offset (6 b).

Address	Tag (hex)	Index	Offset	Result
0x00010004	0x0002	0	4	Miss (cold)
0x00010044	0x0002	1	4	Miss (cold)
0x00010084	0x0002	2	4	Miss (cold)
0x00020004	0x0004	0	4	Miss (conflict – evicts line 0)
0x00010004	0x0002	0	4	Miss (tag mismatch – evicts 0x0001)
0x00010044	0x0002	1	4	Hit (line 1 untouched)

Index computation: $\lfloor 0x10004/64 \rfloor = 1024$; $1024 \bmod 512 = 0$. $\lfloor 0x20004/64 \rfloor = 2048$; $2048 \bmod 512 = 0$ (same index, different tag).

Answer: Direct-mapped: 5 misses, 1 hit

Solution (b) – 4-Way Set-Associative (LRU)

- i. Sets = $512/4 = 128$; index = $\log_2 128 = 7$ bits; tag = $32 - 7 - 6 = 19$ bits.

Total tag bits = $512 \times 19 = 9728$ bits.

- ii. Set = $\lfloor \text{addr}/64 \rfloor \bmod 128$:

Address	Block#	Set	Tag	Result	Set contents
0x00010004	1024	0	2	Miss	S0:{2}
0x00010044	1025	1	2	Miss	S1:{2}
0x00010084	1026	2	2	Miss	S2:{2}
0x00020004	2048	0	4	Miss	S0:{2,4}
0x00010004	1024	0	2	Hit	S0:{4,2} (LRU updated)
0x00010044	1025	1	2	Hit	S1:{2}

Both 0x00010004 and 0x00020004 fit in set 0 simultaneously (4 ways, only 2 used) – no eviction occurs.

Answer: 4-way SA: 4 misses, 2 hits

Solution (c) – Tag Overhead

- Direct-mapped: $512 \times 17 = \mathbf{8704}$ bits
- 4-way SA: $512 \times 19 = \mathbf{9728}$ bits (+1024 bits = 11.8% more)

Answer: 4-way uses 1024 more tag bits than direct-mapped (wider tag to compensate for fewer index bits)

Solution (d) – AMAT

AMAT = hit time + miss rate $\times$ miss penalty

- Direct-mapped: miss rate = $5/6$; AMAT = $2 + (5/6) \times 200 \approx \mathbf{168.7}$ cycles
- 4-way SA: miss rate = $4/6 = 2/3$; AMAT = $2 + (2/3) \times 200 \approx \mathbf{135.3}$ cycles

Answer: 4-way SA is better: 135.3 cycles vs. 168.7 cycles for direct-mapped.

Problem 4

A CPU has a 32 KiB direct-mapped cache with a 128-byte block size. Suppose A is a two-dimensional array of size 512×512 with elements that occupy 8 bytes each. Consider the following two C code segments, $P1$ and $P2$, executed independently with the same initial state (the array A is not in the cache, and variables i, j, x are in registers).

Assumption: Array A is stored in **row-major order** (standard C layout): $A[i][j]$ and $A[i][j+1]$ are adjacent in memory. This is critical for correctly analysing the miss behaviour of $P1$ vs. $P2$.

Listing 1: Code segments $P1$ and $P2$

```
// P1
for (i = 0; i < 512; i++) {
    for (j = 0; j < 512; j++) {
        x += A[i][j];
    }
}

// P2
for (i = 0; i < 512; i++) {
    for (j = 0; j < 512; j++) {
        x += A[j][i];
    }
}
```

Let the number of cache misses experienced by $P1$ be M_1 and that for $P2$ be M_2 . The value of M_1 and M_2 is _____.

Solution – Problem 4

Cache parameters:

- Lines = $32768/128 = 256$; elements per block = $128/8 = 16$
- Total array = $512 \times 512 \times 8 = 2 \text{ MiB} \gg 32 \text{ KiB}$ cache (no reuse across rows)

P1 – row-major access ($A[i][j]$, j varies fastest):

Consecutive accesses are to adjacent memory locations. Each cache block holds 16 consecutive elements. Only 1 miss per 16 elements.

$$M_1 = \frac{512 \times 512}{16} = \frac{262144}{16} = \mathbf{16,384}$$

P2 – column-major access ($A[j][i]$, j varies fastest):

Each step in j jumps by one full row = $512 \times 8 = 4096$ bytes = 32 cache blocks. With only 256 cache lines, after 8 rows the entire cache has been replaced. Every access to $A[j][i]$ loads a new block that will be evicted before it is reused.

$$M_2 = 512 \times 512 = \mathbf{262,144} \quad (\text{every access is a miss})$$

Answer: $M_1 = 16,384$; $M_2 = 262,144$. P1 has $16 \times$ fewer misses than P2.

Problem 5

A **direct-mapped** cache has 8 lines, block size 16 bytes, and uses write-through with no-write-allocate. Memory is byte-addressable.

Address trace (byte addresses, decimal):

0 (read), 4 (write), 16 (read), 32 (write), 48 (write), 16 (write), 64 (read), 80 (read), 0 (write), 16 (read)

(a) Write-Through + No-Write-Allocate

Determine the number of cache hits and misses. Also count how many memory writes are caused by writes and misses (write-through updates memory immediately, so count all writes to memory).

(b) Write-Back + Write-Allocate

Repeat for write-back with write-allocate (assume clean blocks initially, LRU replacement on misses). Count hits, misses, and write-back operations (dirty evictions).

(c) Cycle Cost Comparison

If a memory write costs 100 cycles (for write-through), a cache line fill costs 200 cycles (miss penalty), and write-back only writes dirty blocks on eviction (cost 100 cycles per dirty write), compute the total cycles for each policy using the given address trace.

Solution – Problem 5

Cache line = (address $\div$ 16) mod 8. Blocks 0–5 map to lines 0–5 (all distinct, no conflicts).

Solution (a) – Write-Through + No-Write-Allocate

Rules: read miss $\rightarrow$ allocate; write hit $\rightarrow$ update cache *and* memory; write miss $\rightarrow$ write memory only (no cache allocation).

#	Addr	Op	Line	H/M	Mem writes?
1	0	R	0	M (cold)	0
2	4	W	0	H	1 (write-through)
3	16	R	1	M (cold)	0
4	32	W	2	M (write miss, no alloc)	1 (mem only)
5	48	W	3	M (write miss, no alloc)	1
6	16	W	1	H	1 (write-through)
7	64	R	4	M (cold)	0
8	80	R	5	M (cold)	0
9	0	W	0	H (block 0 from step 1)	1
10	16	R	1	H	0

Answer: Write-Through: 4 hits, 6 misses, 5 memory writes

Solution (b) – Write-Back + Write-Allocate

Rules: read/write miss $\rightarrow$ allocate block, mark dirty on write; evict dirty block $\rightarrow$ write-back.

#	Addr	Op	Line	H/M	Notes
1	0	R	0	M	Load block 0 (clean)
2	4	W	0	H	Mark dirty
3	16	R	1	M	Load block 1 (clean)
4	32	W	2	M	Allocate block 2, mark dirty
5	48	W	3	M	Allocate block 3, mark dirty
6	16	W	1	H	Mark dirty
7	64	R	4	M	Load block 4 (clean)
8	80	R	5	M	Load block 5 (clean)
9	0	W	0	H	Already dirty, update
10	16	R	1	H	Dirty but valid

All 6 accessed blocks map to *different* lines (0–5). No evictions occur $\Rightarrow$ 0 write-backs.

Answer: Write-Back: 4 hits, 6 misses, 0 write-backs

Solution (c) – Cycle cost

Write-through: 6 misses $\times$ 200 + 5 mem-writes $\times$ 100 = 1200 + 500 = **1700** cycles

Write-back: 6 misses $\times$ 200 + 0 write-backs $\times$ 100 = 1200 + 0 = **1200** cycles

Answer: Write-through = 1700 cycles; Write-back = 1200 cycles. Write-back saves 500 cycles.

Problem 6

A small fully associative cache has **4 blocks**, block size **1 word** (simplifying addressing). Memory addresses are word addresses (no offset). The access sequence (in decimal) is:

1, 2, 3, 4, 1, 5, 6, 2, 3, 7, 1, 2, 3, 6

(a) LRU Replacement

Simulate the cache using LRU (Least Recently Used) replacement. Count the number of misses and hits. Show the final contents of the cache.

(b) FIFO Replacement

Simulate the cache using FIFO (First-In-First-Out) replacement. Count the number of

misses and hits. Compare the results with LRU.

(c) Optimal (Belady's) Algorithm

Simulate the optimal (Belady's) algorithm. What is the minimum possible number of misses?

Note: For the Optimal policy, you may show only the **key eviction decisions** at each step. State which block is evicted and why (furthest next use).

(d) Miss Rate and Total Access Time

Calculate the miss rate for each replacement policy (LRU, FIFO, Optimal). If the miss penalty is 100 ns and the hit time is 1 ns, find the total time required to execute the entire address trace under each policy.

Solution – Problem 6

Solution (a) – LRU

State shown as [LRU → MRU]:

Step	Req	H/M	Evicted	Cache [LRU→MRU]
1	1	M	–	[1]
2	2	M	–	[1,2]
3	3	M	–	[1,2,3]
4	4	M	–	[1,2,3,4]
5	1	H	–	[2,3,4,1]
6	5	M	2	[3,4,1,5]
7	6	M	3	[4,1,5,6]
8	2	M	4	[1,5,6,2]
9	3	M	1	[5,6,2,3]
10	7	M	5	[6,2,3,7]
11	1	M	6	[2,3,7,1]
12	2	H	–	[3,7,1,2]
13	3	H	–	[7,1,2,3]
14	6	M	7	[1,2,3,6]

Answer: LRU: 11 misses, 3 hits. Final cache: {1, 2, 3, 6}

Solution (b) – FIFO

Step	Req	H/M	Evicted	Cache [oldest→newest]
1	1	M	–	[1]
2	2	M	–	[1,2]
3	3	M	–	[1,2,3]
4	4	M	–	[1,2,3,4]
5	1	H	–	[1,2,3,4]
6	5	M	1(oldest)	[2,3,4,5]
7	6	M	2	[3,4,5,6]
8	2	M	3	[4,5,6,2]
9	3	M	4	[5,6,2,3]
10	7	M	5	[6,2,3,7]
11	1	M	6	[2,3,7,1]
12	2	H	–	[2,3,7,1]
13	3	H	–	[2,3,7,1]
14	6	M	2	[3,7,1,6]

Answer: FIFO: 11 misses, 3 hits – identical to LRU for this sequence.

Solution (c) – Optimal (Belady's)

At each miss, evict the block whose *next use* is furthest in the future.

Step	Req	H/M	Evict	Reason / Cache after
1	1	M	–	{1}
2	2	M	–	{1,2}
3	3	M	–	{1,2,3}
4	4	M	–	{1,2,3,4}; next uses: $1 \rightarrow 5, 2 \rightarrow 8, 3 \rightarrow 9, 4 \rightarrow \infty$
5	1	H	–	{1,2,3,4}
6	5	M	4	Evict 4 (never used again); {1,2,3,5}; next: $1 \rightarrow 11, 2 \rightarrow 12, 3 \rightarrow 13, 5 \rightarrow \infty$
7	6	M	5	Evict 5 (never again); {1,2,3,6}; next: $1 \rightarrow 11, 2 \rightarrow 12, 3 \rightarrow 13, 6 \rightarrow 14$
8	2	H	–	{1,2,3,6}
9	3	H	–	{1,2,3,6}
10	7	M	6	Evict 6 (step 14); next: $1 \rightarrow 11, 2 \rightarrow 12, 3 \rightarrow 13, 7 \rightarrow \infty$; {1,2,3,7}
11	1	H	–	{1,2,3,7}
12	2	H	–	{1,2,3,7}
13	3	H	–	{1,2,3,7}
14	6	M	7	Evict 7 (never again); {1,2,3,6}

Answer: Optimal: 8 misses, 6 hits – minimum achievable for this trace.

Solution (d) – Miss Rate and Total Time

Policy	Misses	Hits	Miss rate	Total time
LRU	11	3	$11/14 \approx 78.6\%$	$11(100) + 3(1) = 1103$ ns
FIFO	11	3	78.6%	1103 ns
Optimal	8	6	$8/14 \approx 57.1\%$	$8(100) + 6(1) = 806$ ns

Answer: LRU = FIFO = 1103 ns; Optimal = 806 ns (27% faster).

Problem 7

A **direct-mapped** cache has 64 lines, block size 32 bytes. An **automatic next-block prefetcher** is added: on a cache miss, it fetches the requested block and the next consecutive block into the cache (the prefetched block may evict another line). Assume prefetched blocks are placed in the cache according to their address (direct-mapped).

Assumption: Cache replacement follows **direct mapping strictly** for both demand-fetched and prefetched blocks. A prefetched block is placed at the cache line determined by its address, and may evict whatever block currently occupies that line.

Access sequence (byte addresses):

0, 32, 64, 96, 128, 160, 192, 224, 256, 288, 320, 352

(these are all start of blocks 0, 1, 2, ..., 11). Then repeats from 0 again (second iteration).

(a) Without prefetching, compute the miss count for the first iteration and the second iteration.

(b) With prefetching, compute the miss count for the first iteration (prefetcher enabled from the start). Assume the prefetcher does not issue extra requests for prefetched blocks that are already in cache.

(c) For the second iteration (after the first iteration has filled the cache with prefetched blocks), how many misses occur?

(d) Calculate the speedup (execution time reduction) due to prefetching for the two iterations combined, assuming miss penalty = 200 cycles, hit time = 1 cycle, and prefetching adds no overhead (free). Would actual hardware prefetching have any downside?

Solution – Problem 7

Setup: 64 lines, 32-byte blocks. Block n maps to line $n \bmod 64$. Blocks 0–11 map to lines 0–11 (no conflicts among them).

Solution (a) – Without prefetching

1st iteration (cold cache): 12 blocks, all cold misses $\Rightarrow$ **12 misses**. Cache now holds blocks 0–11 in lines 0–11.

2nd iteration: All 12 blocks still present (no conflicts). Every access hits $\Rightarrow$ **0 misses**.

Answer: Without prefetching: 1st = 12 misses; 2nd = 0 misses

Solution (b) – First iteration WITH prefetching

On each miss for block n , the prefetcher also loads block $n + 1$:

Block demanded	In cache?	Result	Prefetched
0	No	Miss	Load block 1 into line 1
1	Yes	Hit	–
2	No	Miss	Load block 3
3	Yes	Hit	–
4	No	Miss	Load block 5
5	Yes	Hit	–
6	No	Miss	Load block 7
7	Yes	Hit	–
8	No	Miss	Load block 9
9	Yes	Hit	–
10	No	Miss	Load block 11
11	Yes	Hit	–

Answer: With prefetching, 1st iteration: 6 misses, 6 hits

Solution (c) – Second iteration WITH prefetching

All blocks 0–11 are in cache after the first iteration. Every demand access hits.

Answer: 2nd iteration: 0 misses

Solution (d) – Speedup

	Misses	Hits	Miss cost	Hit cost
No prefetch (both iters)	12	12	$12 \times 200 = 2400$	$12 \times 1 = 12$
With prefetch (both iters)	6	18	$6 \times 200 = 1200$	$18 \times 1 = 18$

Total without prefetch = 2412 cycles, Total with prefetch = 1218 cycles

$$\text{Speedup} = \frac{2412}{1218} \approx 1.98\times$$

Real-world downsides of hardware prefetching:

- **Cache pollution:** Prefetched blocks may evict useful data.
- **Bandwidth waste:** Prefetches consume memory bus bandwidth even if unused.
- **Harmful for irregular access:** Random/strided patterns cause incorrect predictions.
- **Power overhead:** Extra memory traffic increases energy consumption.

Answer: Speedup $\approx 1.98\times$. Real prefetchers can hurt performance if access patterns are irregular or cache is small.

Problem 8

Given a computing system with two levels of cache (L1 and L2) and a main memory. The first-level (L1) cache access time is 1 nanosecond (ns), and the hit rate for L1 cache is 90% while the processor is accessing the data from L1 cache. For the second-level (L2) cache, the hit rate is 80% and the miss penalty for transferring data from L2 to L1 is 10 ns. The miss penalty for the data to be transferred from main memory to L2 cache is 100 ns. The average memory access time in this system, in nanoseconds, is _____.

Hint: Use the hierarchical AMAT formula: $AMAT = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times t_{mem})$ where m_{L1} = L1 miss rate, m_{L2} = L2 local miss rate.

Solution – Problem 8

Known values: $t_{L1} = 1$ ns, $m_{L1} = 0.10$, $t_{L2} = 10$ ns, $m_{L2} = 0.20$, $t_{mem} = 100$ ns.

$$AMAT = 1 + 0.10 \times (10 + 0.20 \times 100) = 1 + 0.10 \times (10 + 20) = 1 + 0.10 \times 30 = 1 + 3 = 4 \text{ ns}$$

Answer: AMAT = 4 ns

Problem 9

The memory access time is 1 nanosecond for a read operation with a hit in cache, 5 nanoseconds for a read operation with a miss in cache, 2 nanoseconds for a write operation with a hit in cache, and 10 nanoseconds for a write operation with a miss in cache. Execution of a sequence of instructions involves 100 instruction fetch operations, 60 memory operand read operations, and 40 memory operand write operations. The cache hit-ratio is 0.9. The average memory access time (in nanoseconds) in executing the sequence of instructions is _____.

Solution – Problem 9

For each category: $avg = H \cdot t_{hit} + (1 - H) \cdot t_{miss}$, with $H = 0.9$.

Operation	Count	Hit time	Miss time	Avg time
Instruction fetch (read)	100	1 ns	5 ns	$0.9(1) + 0.1(5) = 1.4$ ns
Operand read	60	1 ns	5 ns	1.4 ns
Operand write	40	2 ns	10 ns	$0.9(2) + 0.1(10) = 2.8$ ns

Total time = $100 \times 1.4 + 60 \times 1.4 + 40 \times 2.8 = 140 + 84 + 112 = 336$ ns.

Average per access (= 200 total operations):

$$\overline{T} = \frac{336}{200} = 1.68 \text{ ns}$$

Answer: Total = 336 ns; Average per access = 1.68 ns

Problem 10

A system has a write-through cache with an access time of 100 ns and a hit ratio of 90%. The main memory access time is 1000 ns. 70% of memory references are read operations.

Find the following:

- Average memory access time for read operation only
- Average memory access time for write operation only
- Average memory access time for both operations
- Effective hit ratio

Solution – Problem 10

$t_c = 100$ ns, $t_m = 1000$ ns, $H = 0.9$.

(a) Read AMAT:

$$T_{\text{read}} = H \cdot t_c + (1 - H)(t_c + t_m) = 0.9(100) + 0.1(1100) = 90 + 110 = \mathbf{200} \text{ ns}$$

(b) Write AMAT (write-through always writes to memory):

$$T_{\text{write}} = t_c + t_m = 100 + 1000 = \mathbf{1100} \text{ ns}$$

Every write goes to cache and memory simultaneously; cost is dominated by t_m .

(c) Combined AMAT:

$$T_{\text{avg}} = 0.70 \times 200 + 0.30 \times 1100 = 140 + 330 = \mathbf{470 \text{ ns}}$$

(d) **Effective hit ratio** (fraction of accesses that complete at cache speed):

Only read hits truly avoid main memory. Write hits still go to main memory.

$$H_{\text{eff}} = \text{fraction of reads} \times H = 0.70 \times 0.90 = \mathbf{0.63}$$

Answer: (a) 200 ns; (b) 1100 ns; (c) 470 ns; (d) $H_{\text{eff}} = 0.63$

Problem 11

A two-level cache system has the following parameters:

- **L1:** 16 KiB, direct-mapped, block size 64 B, write-through, no-write-allocate, hit time = 1 ns.
- **L2:** 256 KiB, 8-way set-associative, block size 64 B, write-back, write-allocate, hit time = 10 ns.
- Main memory access time = 100 ns.
- Miss penalty from L2 to memory = 100 ns (plus L2 hit time on a miss? careful with AMAT formula).

Given a sequence of **1024 read accesses** with a **3% global miss rate** (i.e., 3% of all accesses miss in L1). Among those L1 misses, 40% hit in L2.

- (a) Compute the L1 miss rate, L2 miss rate (local to L2), and global L2 miss rate.
- (b) Compute the Average Memory Access Time (AMAT) for this system.
- (c) If we change L1 to write-back + write-allocate, the hit time becomes 1.2 ns (due to extra complexity). Assume the write traffic to L2 is reduced by 50% (affecting L2 miss rate? No, only reads considered here – but assume read miss rate same). Would the total AMAT improve? Explain briefly.
- (d) What is the effective memory access time for a write operation under the original L1 write-through policy? (Assume writes that hit L1 go to L2 as well, and L2 hit time is the same as L1 miss penalty to L2.)

Solution – Problem 11

Solution (a) – Miss rates

- L1 miss rate: $m_{L1} = 3\% = 0.03$
- L2 local miss rate: $m_{L2} = 1 - 0.40 = 0.60$ (60% of L1 misses also miss L2)
- Global L2 miss rate: $m_{L1} \times m_{L2} = 0.03 \times 0.60 = 0.018 = 1.8\%$

Answer: $m_{L1} = 3\%$; $m_{L2\text{local}} = 60\%$; **global L2 miss rate** = 1.8%

Solution (b) – AMAT

$$\text{AMAT} = 1 + 0.03 \times (10 + 0.60 \times 100) = 1 + 0.03 \times 70 = 1 + 2.1 = \mathbf{3.1 \text{ ns}}$$

Answer: **AMAT = 3.1 ns**

Solution (c) – Write-back L1 comparison

New AMAT = $1.2 + 0.03 \times 70 = 1.2 + 2.1 = 3.3 \text{ ns} > 3.1 \text{ ns}$.

AMAT *worsens*. The extra 0.2 ns hit-time penalty on every access outweighs the reduction in write traffic, which does not directly affect read AMAT.

Answer: **AMAT increases to 3.3 ns. Write-back L1 is worse for this workload.**

Solution (d) – Write AMAT (write-through L1)

Every write hits L1 (1 ns) and is forwarded to L2 (10 ns). On an L2 miss (60% of the 3% L1 misses) the write also goes to main memory.

$$T_W = 1 + 10 + m_{L1} \times m_{L2} \times t_{mem} = 11 + 0.03 \times 0.60 \times 100 = 11 + 1.8 = \mathbf{12.8 \text{ ns}}$$

Answer: **Write AMAT = 12.8 ns**

Problem 12

A cache has the following miss rates for different block sizes (given fixed total cache size, 4-way set-associative):

Block size (bytes)	Miss rate	Miss penalty (cycles)
16	0.10	40
32	0.07	70
64	0.05	130
128	0.045	250

Hit time = 2 cycles for all block sizes.

- (a) Compute AMAT for each block size. Which block size is optimal?
- (b) If the cache is **direct-mapped** instead, miss rates would increase by 20% (multiply by 1.2) for each block size. Recalculate AMAT. Does the optimal block size change?
- (c) Suppose the miss penalty includes the transfer time of the block (proportional to block size). For 32-byte block, penalty = 70. For 64-byte, penalty = 130. That is roughly $2 \times$ block size gives $1.86 \times$ penalty. Derive a linear model: $\text{penalty} = a + b \times \text{block_size}$. Then find the block size that minimizes AMAT given miss rate = $c \times \sqrt{\text{block_size}}$ (assume $c = 0.2$ for 16B, adjust). (This is a calculus or trial numeric problem.)

Solution – Problem 12

Solution (a) – 4-Way SA: $\text{AMAT} = 2 + r \times p$

Block	r	p	$r \times p$	AMAT
16 B	0.10	40	4.00	6.00
32 B	0.07	70	4.90	6.90
64 B	0.05	130	6.50	8.50
128 B	0.045	250	11.25	13.25

Answer: Optimal block size = 16 bytes (AMAT = 6.0 cycles). Larger blocks increase penalty faster than they reduce miss rate.

Solution (b) – Direct-Mapped (miss rates $\times 1.2$)

Block	r (new)	p	$r \times p$	AMAT
16 B	0.120	40	4.80	6.80
32 B	0.084	70	5.88	7.88
64 B	0.060	130	7.80	9.80
128 B	0.054	250	13.50	15.50

Answer: Optimal block size still = 16 bytes (AMAT = 6.8 cycles). The ranking does not change.

Solution (c) – Linear penalty model

Fit $p = a + bB$ using data points (32, 70) and (64, 130):

$$70 = a + 32b, \quad 130 = a + 64b \Rightarrow 60 = 32b \Rightarrow b = 1.875, \quad a = 10$$

Model: $p(B) = 10 + 1.875B$.

Miss rate: $r(B) = c\sqrt{B}$ with $r(16) = 0.10 \Rightarrow c = 0.10/4 = 0.025$.

$$\text{AMAT}(B) = 2 + 0.025\sqrt{B} \cdot (10 + 1.875B)$$

Taking $\frac{d}{dB}(\text{AMAT}) = 0$:

$$0.025 \left(\frac{5}{\sqrt{B}} + \frac{3 \times 1.875}{2} \sqrt{B} \right) = 0 \Rightarrow \frac{5}{\sqrt{B}} = -2.8125\sqrt{B}$$

No positive solution exists, confirming AMAT is strictly increasing in B for this model.

Answer: Under this model, AMAT is monotonically increasing – the smallest block size always minimises AMAT, consistent with parts (a) and (b).